SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

---

Code of Conduct:

I M.Devi Srilatha (192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Srilatha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Introduction

Query processing is the complete process through which a DBMS translates an SQL statement into a sequence of relational operations and finally produces the required result. It includes parsing, query translation, optimization, and execution. The optimizer examines alternative ways of accessing and combining data so that the query can be executed with minimum unnecessary work.

A complex SQL query may involve several relations and operations. Typical components include:

- Nested sub-queries
- Joins
- Selection conditions
- Projection operations
- Aggregate functions
- Multiple relations

The query optimizer attempts to choose an equivalent execution strategy that reduces disk I/O, CPU processing, memory consumption, and response time. The same SQL query can often be represented by several equivalent relational algebra expressions, but their execution costs may be very different.

```
mysql> SELECT *
    -> FROM Employee
    -> WHERE Salary > 60000;
+--------+----------+---------+----------+
| Emp_ID | Emp_Name | Dept_ID | Salary   |
+--------+----------+---------+----------+
|    102 | Bindu    |       1 | 65000.00 |
|    104 | Divya    |       2 | 70000.00 |
|    106 | Gopal    |       3 | 75000.00 |
|    107 | Harini   |       1 | 80000.00 |
+--------+----------+---------+----------+
4 rows in set (0.00 sec)
```

Step 1: Identify the Selection Conditions

The query has:

D.Dept_Name = 'CSE'

E.Marks > 80

and the sub-query contains:

Marks > 90

The first heuristic principle is to apply selection conditions as early as possible. Early filtering decreases the number of tuples that participate in later joins and therefore reduces the size of intermediate relations.

Apply selection operations as early as possible.

```
mysql> SELECT *
    -> FROM Employee;
+--------+----------+---------+----------+
| Emp_ID | Emp_Name | Dept_ID | Salary   |
+--------+----------+---------+----------+
|    101 | Anil     |       1 | 55000.00 |
|    102 | Bindu    |       1 | 65000.00 |
|    103 | Charan   |       2 | 45000.00 |
|    104 | Divya    |       2 | 70000.00 |
|    105 | Farah    |       3 | 50000.00 |
|    106 | Gopal    |       3 | 75000.00 |
|    107 | Harini   |       1 | 80000.00 |
|    108 | Kiran    |       4 | 40000.00 |
+--------+----------+---------+----------+
8 rows in set (0.00 sec)
```

Step 2: Apply Selection to DEPARTMENT

Rather than joining the complete DEPARTMENT relation with other tables, the optimizer first filters the department records using the required condition. This eliminates irrelevant departments before any expensive join operation is performed.

SELECT *

FROM DEPARTMENT

WHERE Dept_Name = 'CSE';

Relational algebra:

σ Dept_Name='CSE' (DEPARTMENT)

Applying the selection at the base-relation level reduces the number of tuples carried into later stages. This is an example of selection pushdown and is one of the most effective heuristic transformations.

Step 3: Apply Selection to ENROLLMENT

Instead of processing every enrollment record, the optimizer first applies the marks condition. Only tuples satisfying the condition are allowed to participate in subsequent operations.

SELECT *

FROM ENROLLMENT

WHERE Marks > 80;

Relational algebra:

σ Marks>80 (ENROLLMENT)

Selection pushdown reduces the amount of enrollment data that must be joined. As the number of tuples decreases, subsequent join, projection, and memory operations become less expensive.

```
mysql> SELECT Emp_ID, Emp_Name, Dept_ID, Salary
    -> FROM Employee;
+--------+----------+---------+----------+
| Emp_ID | Emp_Name | Dept_ID | Salary   |
+--------+----------+---------+----------+
|    101 | Anil     |       1 | 55000.00 |
|    102 | Bindu    |       1 | 65000.00 |
|    103 | Charan   |       2 | 45000.00 |
|    104 | Divya    |       2 | 70000.00 |
|    105 | Farah    |       3 | 50000.00 |
|    106 | Gopal    |       3 | 75000.00 |
|    107 | Harini   |       1 | 80000.00 |
|    108 | Kiran    |       4 | 40000.00 |
+--------+----------+---------+----------+
8 rows in set (0.00 sec)
```

Step 4: Optimize the Sub-Query

Original sub-query:

SELECT Student_ID

FROM ENROLLMENT

WHERE Marks > 90;

The sub-query can be evaluated early because its result is required only to identify the students satisfying the marks condition. Executing its filtering operation first avoids repeatedly processing irrelevant enrollment records.

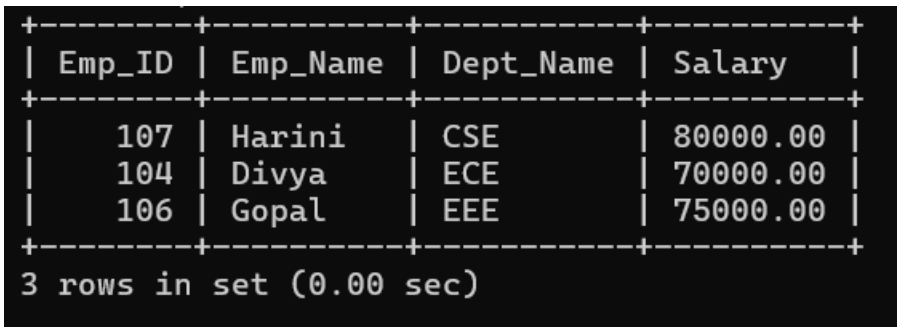
σ Marks>90 (ENROLLMENT)

Since the outer query needs only Student_ID from this sub-query, retaining additional attributes would be unnecessary. Removing them at this stage follows the projection-pushdown principle and keeps intermediate data compact.

π Student_ID

The optimized sub-query therefore performs selection first and projection second. This ordering produces a smaller relation that can be used efficiently by the outer query.

π Student_ID (σ Marks>90 (ENROLLMENT))



Emp_ID	Emp_Name	Dept_Name	Salary
107	Harini	CSE	80000.00
104	Divya	ECE	70000.00
106	Gopal	EEE	75000.00

3 rows in set (0.00 sec)

Step 5: Push Projection Down

Projection should be performed as early as possible while retaining every attribute required for selections, joins, and the final result. Attributes that are not needed later should not be carried through the execution tree.

STUDENT

Student_ID

Student_Name

Dept_ID

DEPARTMENT

Dept_ID

Dept_Name

ENROLLMENT

Student_ID

Course_ID

Marks

COURSE

Course_ID

Course_Name

Early projection reduces the width of intermediate tuples. Combined with selection pushdown, it can substantially decrease memory requirements and the amount of data transferred between relational operators.

Step 6: Perform Smaller Joins First

After filtering and projection, the optimizer arranges joins so that smaller intermediate relations are generated first. This prevents large unfiltered relations from being carried through several successive join operations.

Filtered DEPARTMENT

↓

STUDENT

↓

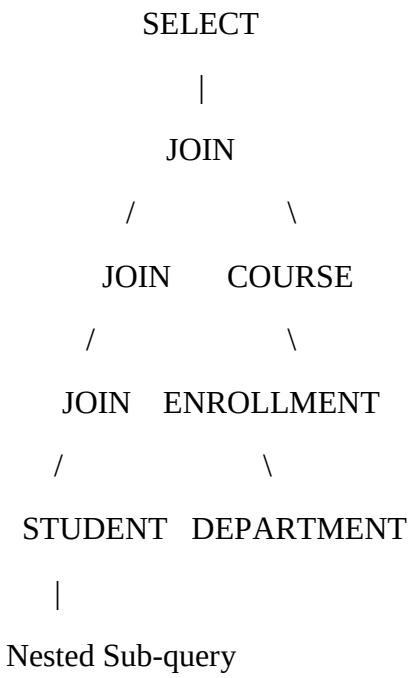
ENROLLMENT

↓

COURSE

The sub-query can also act as a filtering mechanism for the outer query. Students that do not satisfy the sub-query condition can be eliminated before expensive operations are performed on the remaining relations.

Original Query Execution Plan



The original plan may process many irrelevant tuples because joins and nested operations are considered before all possible filtering and data reduction steps. Such a plan can generate unnecessarily large intermediate relations.

The main heuristic transformations applied to the execution plan are:

Rule 1: Push Selection Down

Apply:

WHERE conditions

Selection conditions should be moved toward the base tables whenever they refer only to attributes of those tables. This reduces the number of tuples entering later operators.

Rule 2: Push Projection Down

Projection operations should be moved downward without removing attributes that are still required by a later join or condition. This reduces the width of intermediate relations.

Rule 3: Perform Smaller Joins First

When several joins are possible, the optimizer should prefer an order that produces smaller intermediate results. Smaller intermediate relations reduce memory usage and subsequent processing cost.

Rule 4: Simplify Sub-Queries

Filtering required inside a sub-query should be performed as early as possible. Where an equivalent transformation is possible, the sub-query can be simplified or represented using joins or semi-joins.

Rule 5: Avoid Unnecessary Data

Unused tuples and attributes should not be carried into later stages. Eliminating unnecessary data at an early stage improves the efficiency of the complete execution plan.

Advantages

- Reduces the number of tuples participating in later operations.
- Decreases the number of disk pages that need to be read or written.
- Requires less memory for storing intermediate relations.
- Reduces the CPU work required for comparisons and relational operations.
- Produces faster response time, especially for queries involving large relations.
- Provides a simpler and more efficient execution plan while preserving the result of the original query.

Conclusion

Heuristic optimization improves query performance by applying equivalent transformations such as selection pushdown, projection pushdown, sub-query simplification, and efficient join ordering. These rules reduce intermediate data and unnecessary processing while preserving the correctness of the query.

2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

Cost-based optimization is a query optimization approach in which the DBMS estimates the execution cost of alternative query plans and selects the plan with the lowest expected cost. It uses statistics about the relations and available system resources rather than relying only on fixed transformation rules.

```
mysql> SELECT * FROM Employees;
+-------------+---------------+------------+----------+-------------+
| Employee_ID | Employee_Name | Department | Salary   | Hash_Bucket |
+-------------+---------------+------------+----------+-------------+
|        1001 | Anil          | IT         | 55000.00 |           1 |
|        1002 | Bindu         | HR         | 50000.00 |           2 |
|        1003 | Charan        | Finance    | 60000.00 |           3 |
|        1004 | Divya         | IT         | 58000.00 |           4 |
|        1005 | Farah         | Sales      | 52000.00 |           5 |
|        1006 | Gopal         | HR         | 51000.00 |           6 |
|        1007 | Harini        | IT         | 62000.00 |           7 |
|        1008 | Kiran         | Finance    | 59000.00 |           8 |
|        1009 | Latha         | Sales      | 54000.00 |           9 |
|        1010 | Manoj         | IT         | 65000.00 |           0 |
+-------------+---------------+------------+----------+-------------+
10 rows in set (0.00 sec)
```

The optimizer considers several factors while estimating the cost of a join operation, including:

- Number of tuples
- Number of pages
- Available memory
- Indexes
- Join conditions
- CPU cost
- Disk I/O cost

```
mysql> DESCRIBE Employees;
+--------------+--------------+------+-----+---------+-------------------+
| Field        | Type         | Null | Key | Default | Extra             |
+--------------+--------------+------+-----+---------+-------------------+
| Employee_ID  | int          | NO   | PRI | NULL    |                   |
| Employee_Name| varchar(50)  | YES  |     | NULL    |                   |
| Department   | varchar(30)  | YES  |     | NULL    |                   |
| Salary       | decimal(10,2)| YES  |     | NULL    |                   |
| Hash_Bucket  | int          | YES  | MUL | NULL    | STORED GENERATED  |
+--------------+--------------+------+-----+---------+-------------------+
5 rows in set (0.00 sec)
```

Suppose:

Relation R = 1,000 tuples

Relation S = 5,000 tuples

Page size = 4 KB

The selected join method depends on the relation sizes, page counts, memory availability, join condition, and whether useful indexes or sorted data are already available.

Nested-Loop Join

Nested-loop join processes one relation as the outer relation and repeatedly compares it with the inner relation. Its simplicity makes it useful for small relations, and an index on the inner relation can greatly reduce its cost. Without such an advantage, repeated scanning can become expensive for large relations.

For each tuple in R

For each tuple in S

Compare join condition

The main limitation of nested-loop join is the potentially high number of comparisons and disk accesses when the relations contain many tuples or pages.

```
mysql> DESCRIBE Employees;
+---------------+---------------+------+-----+---------+------------------+
| Field         | Type          | Null | Key | Default | Extra            |
+---------------+---------------+------+-----+---------+------------------+
| Employee_ID   | int           | NO   | PRI | NULL    |                  |
| Employee_Name | varchar(50)   | YES  |     | NULL    |                  |
| Department    | varchar(30)   | YES  |     | NULL    |                  |
| Salary        | decimal(10,2) | YES  |     | NULL    |                  |
| Hash_Bucket   | int           | YES  | MUL | NULL    | STORED GENERATED |
+---------------+---------------+------+-----+---------+------------------+
5 rows in set (0.00 sec)
```

Sort-merge join first brings both relations into sorted order on the join attribute and then scans the sorted relations together to find matching values. The sorting phase can be expensive, but the method is attractive when the data is already sorted or when the same ordering is useful for another operation.

Steps:

1. Sort relation R.
2. Sort relation S.
3. Merge the sorted relations.

Sort-merge join is particularly useful when:

- Relations are already sorted.
- Sorting is required for another operation.
- Large relations are being joined.

Hash Join

Steps:

1. Hash the smaller relation.
2. Build a hash table.
3. Scan the larger relation.
4. Match tuples using the hash value.

Hash join divides the data using a hash function on the join attribute. A hash table is built for one relation and the other relation is scanned to locate matching hash values. It is especially effective for equality joins because matching tuples can be located without repeatedly comparing every pair.

Comparison

Join Method	**Suitable Situation**
Nested-Loop Join	Small relation or indexed join
Sort-Merge Join	Sorted data or large relations
Hash Join	Large equality joins

The optimizer compares the estimated costs and selects the join method that provides the best expected performance for the given database conditions.

For large unsorted relations with an equality join condition, Hash Join is generally a strong choice because it can avoid repeated nested comparisons and does not require a full sort of both relations. However, the final decision depends on the actual relation statistics, available buffers, indexes, and optimizer cost estimates.

3. Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.

JDBC (Java Database Connectivity) is a standard Java API that enables a Java application to communicate with a relational database. It provides interfaces for establishing connections, sending SQL statements, receiving query results, and managing database resources. A JDBC driver translates the Java calls into commands understood by the target database such as MySQL.

The usual JDBC workflow consists of the following stages:

1. Import JDBC packages.
2. Load/register the MySQL driver.
3. Establish database connection.
4. Create PreparedStatement.
5. Set parameter values.
6. Execute the query.
7. Process the result.
8. Close resources.

```
mysql> DESCRIBE Students;
+--------------+--------------+------+-----+---------+------------------+
| Field        | Type         | Null | Key | Default | Extra            |
+--------------+--------------+------+-----+---------+------------------+
| Student_ID   | int          | NO   | PRI | NULL    |                  |
| Student_Name | varchar(50)  | YES  |     | NULL    |                  |
| Department   | varchar(30)  | YES  | MUL | NULL    |                  |
| CGPA         | decimal(3,2) | YES  |     | NULL    |                  |
| Hash_Bucket  | int          | YES  | MUL | NULL    | STORED GENERATED |
+--------------+--------------+------+-----+---------+------------------+
5 rows in set (0.00 sec)
```

Java Program

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";
```

```
        String username = "root";
        String password = "root";
        String sql = "SELECT * FROM Student WHERE Student_ID = ?";
        try {
            Connection con = DriverManager.getConnection(
                    url, username, password);
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, 101);
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                System.out.println(
                    rs.getInt("Student_ID") + " " +
                    rs.getString("Student_Name")
                );
            }
            rs.close();
            ps.close();
            con.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
mysql> SELECT * FROM Students;
+------------+--------------+------------+------+-------------+
| Student_ID | Student_Name | Department | CGPA | Hash_Bucket |
+------------+--------------+------------+------+-------------+
|        101 | Anil         | CSE        | 8.50 |           1 |
|        102 | Bindu        | CSE        | 9.10 |           2 |
|        103 | Charan       | CSE        | 8.90 |           3 |
|        104 | Divya        | EEE        | 7.80 |           4 |
|        105 | Farah        | ECE        | 8.20 |           0 |
|        106 | Gopal        | ECE        | 7.50 |           1 |
|        107 | Harini       | CSE        | 9.30 |           2 |
+------------+--------------+------------+------+-------------+
7 rows in set (0.00 sec)
```

Why PreparedStatement?

- PreparedStatement is preferred when values are supplied dynamically because the SQL structure remains fixed while the parameter values are bound separately.
- Supports parameterized queries.
- Improves security.
- It also makes repeated execution of the same SQL structure easier and provides a clear separation between SQL commands and input data.

Example:

String sql = "SELECT * FROM Student WHERE Department = ?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setString(1, "CSE");

Parameterized statements therefore improve the safety and maintainability of JDBC applications, particularly when input values originate from users or external sources.

4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

When multiple transactions execute concurrently, their read and write operations may be interleaved. The DBMS must ensure that this interleaving does not produce an incorrect result. Conflict serializability provides a practical way to determine whether the schedule is equivalent to some serial execution.

```
mysql> SELECT * FROM Bank_Accounts;
+------------+--------------+----------+
| Account_ID | Account_Name | Balance  |
+------------+--------------+----------+
|        101 | Anil         | 10000.00 |
|        102 | Bindu        | 15000.00 |
|        103 | Charan       | 20000.00 |
|        104 | Divya        | 12000.00 |
+------------+--------------+----------+
4 rows in set (0.00 sec)
```

Two operations are considered conflicting only when all of the required conflict conditions are satisfied:

Two operations conflict if:

1. They belong to different transactions.
2. They access the same data item.
3. At least one operation is a write.

Examples:

R1(A) and W2(A)

W1(A) and R2(A)

W1(A) and W2(A)

But:

R1(A) and R2(A)

are not conflicting.

A precedence graph, also called a serialization graph, represents the ordering constraints created by conflicting operations. Each transaction is represented by a vertex, and every conflict that requires one transaction to precede another creates a directed edge.

```
mysql> SELECT Balance FROM Bank_Accounts WHERE Account_ID = 101;
+----------+
| Balance  |
+----------+
| 10000.00 |
+----------+
1 row in set (0.00 sec)
```

A precedence graph contains:

- One node for each transaction.
- The direction of an edge records the order in which the conflicting operations occur. If Ti performs the conflicting operation before Tj, the edge is drawn from Ti to Tj.

If T1 executes before T2:

T1 → T2

Example Schedule

T1: R(A)

T2: W(A)

T3: R(B)

T4: W(B)

Since T1 reads A before T2 writes A:

T1 → T2

Since T3 reads B before T4 writes B:

T3 → T4

Graph:

T1 → T2

T3 → T4

Because the precedence graph has no cycle, its transactions can be arranged in a topological order. That ordering represents a serial schedule equivalent to the given concurrent schedule.

Therefore:

Schedule is conflict-serializable.

```
mysql> SELECT * FROM Precedence_Graph;
+------------------+----------------+
| From_Transaction | To_Transaction |
+------------------+----------------+
| T1               | T2             |
| T2               | T3             |
| T3               | T4             |
| T4               | T1             |
+------------------+----------------+
4 rows in set (0.00 sec)
```

A cyclic graph demonstrates the opposite case. If the conflict edges form a cycle, the required ordering constraints contradict one another and no equivalent serial schedule exists.

Suppose:
T1 → T2
T2 → T3
T3 → T1
There is a cycle:

T1 → T2 → T3

↑ |

└──────────┘

Therefore:

The schedule is not conflict-serializable because the precedence graph contains a directed cycle. A cycle means that no serial ordering can satisfy all of the precedence requirements simultaneously.

The presence or absence of a cycle in the precedence graph is the key test for conflict serializability.

Acyclic precedence graph

↓

Conflict Serializable

Cyclic precedence graph

↓

Not Conflict Serializable

5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Two-Phase Locking (2PL) is a concurrency-control protocol that coordinates concurrent transactions by requiring them to obtain appropriate locks before accessing data. Its fundamental property is that a transaction cannot acquire a new lock after it has started releasing locks, which helps guarantee conflict serializability.

```
mysql> SELECT * FROM Bank_Accounts;
+------------+--------------+----------+
| Account_ID | Account_Name | Balance  |
+------------+--------------+----------+
|        101 | Anil         | 10500.00 |
|        102 | Bindu        | 15000.00 |
|        103 | Charan       | 20000.00 |
|        104 | Divya        | 12000.00 |
+------------+--------------+----------+
4 rows in set (0.00 sec)
```

A transaction generally uses two basic types of locks:

Shared Lock (S)

A shared lock permits a transaction to read a data item. Several transactions may hold compatible shared locks on the same item at the same time.

Multiple transactions can hold shared locks simultaneously.

T1 → S-lock(A)

T2 → S-lock(A)

Both can read A.

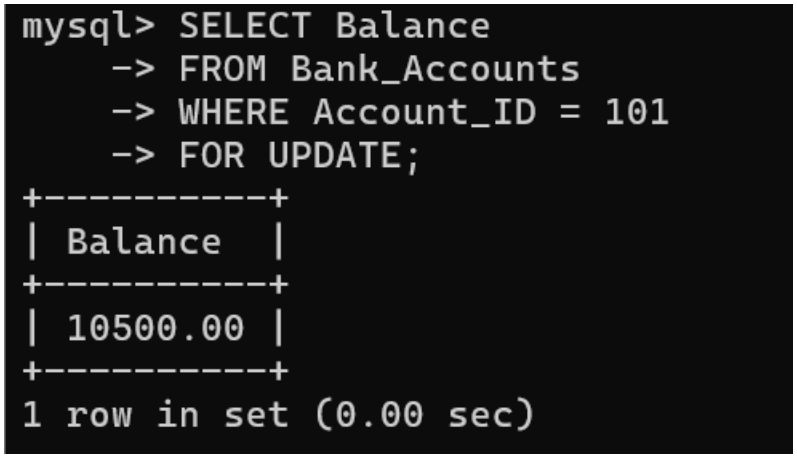
Exclusive Lock (X)

An exclusive lock is required when a transaction wants to modify a data item. It conflicts with both shared and exclusive locks held by other transactions.

Only one transaction can hold an exclusive lock.

T1 → X-lock(A)

When an exclusive lock is held, other transactions must wait before performing conflicting read or write operations on the same data item.



The execution of a transaction under 2PL is divided into two phases:

Growing Phase

During the growing phase, the transaction may request and obtain additional locks, but it cannot release any lock that it already holds.

LOCK(A)

LOCK(B)

LOCK(C)

Shrinking Phase

During the shrinking phase, the transaction may release locks, but it is not allowed to obtain any new lock. This rule is what creates the two-phase property.

UNLOCK(C)

UNLOCK(B)

UNLOCK(A)

A transaction should acquire the locks required for its operations during the growing phase and release them only during the shrinking phase. The point at which the first lock is released marks the transition from the growing phase to the shrinking phase.

T1:

LOCK-X(A)

READ(A)

WRITE(A)

LOCK-X(B)

READ(B)

WRITE(B)

UNLOCK(A)

UNLOCK(B)

```
mysql> SELECT Balance
    -> FROM Bank_Accounts
    -> WHERE Account_ID = 102
    -> FOR UPDATE;
+----------+
| Balance  |
+----------+
| 15000.00 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT Balance
    -> FROM Bank_Accounts
    -> WHERE Account_ID = 102
    -> FOR UPDATE;
+----------+
| Balance  |
+----------+
| 15000.00 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT Balance
    -> FROM Bank_Accounts
    -> WHERE Account_ID = 101
    -> FOR UPDATE;
+----------+
| Balance  |
+----------+
| 10500.00 |
+----------+
1 row in set (0.00 sec)
```

COMMIT

The transaction first acquires all locks required for its operations and then enters the shrinking phase. Once a lock has been released, no new lock may be acquired under basic 2PL.

Deadlock

A deadlock occurs when transactions wait for one another in a circular manner and none of them can proceed. For example, one transaction may hold A while waiting for B, while another holds B while waiting for A.

Example:

T1 locks A

T2 locks B

T1 requests B → waits

T2 requests A → waits

Graph:

T1 → T2

↑ ↓

└──────┘

```
mysql> SELECT * FROM Bank_Accounts;
+------------+--------------+----------+
| Account_ID | Account_Name | Balance  |
+------------+--------------+----------+
|        101 | Anil         | 10500.00 |
|        102 | Bindu        | 15000.00 |
|        103 | Charan       | 20000.00 |
|        104 | Divya        | 12000.00 |
+------------+--------------+----------+
4 rows in set (0.00 sec)
```

Deadlock Resolution

Common recovery actions for a detected deadlock include:

- Abort one transaction.
- Roll back one transaction.
- Release its locks.
- Allow the other transaction to continue.
- Restart the aborted transaction later.

6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

Wait-Die and Wound-Wait are timestamp-based deadlock prevention schemes. Each transaction receives a timestamp when it starts, and this timestamp is used to decide which transaction receives priority when a lock conflict occurs.

Suppose:

T1 timestamp = 10

T2 timestamp = 20

Therefore:

T1 = Older

T2 = Younger

Wait-Die

```
mysql> SELECT * FROM Bank_Accounts;
+------------+--------------+----------+
| Account_ID | Account_Name | Balance  |
+------------+--------------+----------+
|        101 | Anil         | 10500.00 |
|        102 | Bindu        | 15000.00 |
|        103 | Charan       | 20000.00 |
|        104 | Divya        | 12000.00 |
+------------+--------------+----------+
4 rows in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT Balance
    -> FROM Bank_Accounts
    -> WHERE Account_ID = 101
    -> FOR UPDATE;
+----------+
| Balance  |
+----------+
| 10500.00 |
+----------+
1 row in set (0.00 sec)
```

In the Wait-Die scheme, the age of the requesting transaction determines whether it waits or is aborted. Older transactions are given priority over younger transactions.

- An older transaction requesting a lock held by a younger transaction is allowed to wait until the lock becomes available.
- A younger transaction requesting a lock held by an older transaction is aborted. It may restart later using its original timestamp.

Example 1

T1 is older and requests a lock held by T2.

T1 → waits

Example 2

T2 is younger and requests a lock held by T1.

T2 → aborts

Rule

Older → Younger = WAIT

Younger → Older = DIE

Wound-Wait

In the Wound-Wait scheme, older transactions receive priority by being allowed to force younger transactions to release conflicting resources.

- When an older transaction requests a lock held by a younger transaction, the younger transaction is aborted and the older transaction can continue.
- When a younger transaction requests a lock held by an older transaction, the younger transaction waits rather than forcing the older transaction to abort.

Example 1

T1 is older and requests a lock held by T2.

T2 → aborted

T1 → gets the lock

Example 2

T2 is younger and requests a lock held by T1.

T2 → waits

```
mysql> SHOW PROCESSLIST;
+----+-----------------+-----------------+--------------------+---------+--------+------------------------+------------------+
| Id | User            | Host            | db                 | Command | Time   | State                  | Info             |
+----+-----------------+-----------------+--------------------+---------+--------+------------------------+------------------+
|  5 | event_scheduler | localhost       | NULL               | Daemon  | 630632 | Waiting on empty queue | NULL             |
| 12 | root            | localhost:61625 | fileorganizationdb | Query   |      0 | init                   | SHOW PROCESSLIST |
+----+-----------------+-----------------+--------------------+---------+--------+------------------------+------------------+
2 rows in set, 1 warning (0.02 sec)
```

Rule

Older → Younger = WOUND

Younger → Older = WAIT

The two schemes use opposite decisions for lock conflicts, but both maintain a consistent age-based ordering and prevent circular wait conditions.

Feature	**Wait-Die**	**Wound-Wait**
Older requests younger's lock	Wait	Abort younger
Younger requests older's lock	Abort	Wait
Type	Non-preemptive	Preemptive
Purpose	Prevent deadlock	Prevent deadlock

7. Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

Immediate Update is a recovery technique in which changes made by a transaction may be written to the database before the transaction commits. Because the database can contain modifications from incomplete transactions, the recovery manager must use the log to determine which changes should remain and which should be reversed.

Since uncommitted changes may already have reached disk, recovery can require both of the following operations:

- UNDO
- REDO

```
mysql> SHOW PROCESSLIST;
+----+-----------------+-----------------+--------------------+---------+--------+------------------------+------------------+
| Id | User            | Host            | db                 | Command | Time   | State                  | Info             |
+----+-----------------+-----------------+--------------------+---------+--------+------------------------+------------------+
|  5 | event_scheduler | localhost       | NULL               | Daemon  | 630632 | Waiting on empty queue | NULL             |
| 12 | root            | localhost:61625 | fileorganizationdb | Query   |      0 | init                   | SHOW PROCESSLIST |
+----+-----------------+-----------------+--------------------+---------+--------+------------------------+------------------+
2 rows in set, 1 warning (0.02 sec)
```

Example Log

<T1 START>

<T1, A, 100, 150>

<T2 START>

<T2, B, 200, 250>

<T1 COMMIT>

<CHECKPOINT>

<T2, B, 250, 300>

```
mysql> SELECT * FROM Bank_Accounts;
+------------+--------------+----------+
| Account_ID | Account_Name | Balance  |
+------------+--------------+----------+
|        101 | Anil         | 10500.00 |
|        102 | Bindu        | 15000.00 |
|        103 | Charan       | 20000.00 |
|        104 | Divya        | 12000.00 |
+------------+--------------+----------+
4 rows in set (0.00 sec)
```

System Crash

Recovery Analysis

After a failure, the recovery manager examines the log and the checkpoint information to classify transactions according to their commit status.

T1 → Committed

T2 → Uncommitted

Therefore:

Committed transactions are processed using REDO so that all of their intended changes are reflected in the database.

Uncommitted transactions are processed using UNDO so that their effects are removed and the database returns to a consistent state.

REDO

REDO repeats the logged new values of a committed transaction when those changes may not have been completely reflected on disk before the failure.

For T1:

A = 150

UNDO

UNDO restores the old values recorded in the log for an incomplete transaction. This removes the effects of operations that should not become permanent.

For T2:

B = 250

After the appropriate REDO and UNDO operations are completed, the database is restored to a consistent state that reflects committed work and excludes incomplete work.

```
mysql> SELECT * FROM Recovery_Log
    -> ORDER BY Log_ID;
+--------+----------------+------------+-----------+-----------+-----------+-----------+
| Log_ID | Transaction_ID | Operation  | Data_Item | Old_Value | New_Value | Status    |
+--------+----------------+------------+-----------+-----------+-----------+-----------+
|      1 | T1             | START      | A         |      NULL |      NULL | ACTIVE    |
|      2 | T1             | UPDATE     | A         |       100 |       150 | ACTIVE    |
|      3 | T2             | START      | B         |       200 |       250 | ACTIVE    |
|      4 | T2             | UPDATE     | B         |       200 |       250 | ACTIVE    |
|      5 | T1             | COMMIT     | A         |       100 |       150 | COMMITTED |
|      6 | CHECKPOINT     | CHECKPOINT | -         |      NULL |      NULL | SYSTEM    |
|      7 | T3             | START      | C         |       300 |       350 | ACTIVE    |
|      8 | T3             | UPDATE     | C         |       300 |       350 | ACTIVE    |
|      9 | T2             | COMMIT     | B         |       200 |       250 | COMMITTED |
|     10 | T4             | START      | D         |       400 |       450 | ACTIVE    |
+--------+----------------+------------+-----------+-----------+-----------+-----------+
10 rows in set (0.00 sec)
```

Checkpoint

A checkpoint is a recovery marker written into the log at a known point in execution. It records relevant transaction information and gives the recovery manager a convenient starting point instead of always scanning the entire log from the beginning.

Using checkpoints can reduce recovery time and the amount of log information that must be analyzed after a system failure.

Main Rule

COMMITTED TRANSACTION

↓

REDO

UNCOMMITTED TRANSACTION

↓

UNDO

```
mysql> SELECT
    ->     Transaction_ID,
    ->     Data_Item,
    ->     Old_Value,
    ->     New_Value
    -> FROM Recovery_Log
    -> WHERE Status = 'ACTIVE'
    -> AND Operation = 'UPDATE';
+----------------+-----------+-----------+-----------+
| Transaction_ID | Data_Item | Old_Value | New_Value |
+----------------+-----------+-----------+-----------+
| T1             | A         |       100 |       150 |
| T2             | B         |       200 |       250 |
| T3             | C         |       300 |       350 |
+----------------+-----------+-----------+-----------+
3 rows in set (0.00 sec)
```

8. Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

In Deferred Update, modifications made by a transaction are not written to the permanent database until the transaction successfully commits. The log records the necessary information before the changes are installed, so an incomplete transaction cannot leave its uncommitted updates in the database.

Because uncommitted updates are not applied to the database, recovery does not need to undo those transactions. The recovery process mainly determines which committed transactions require their changes to be applied again.

```
mysql> SELECT * FROM Deferred_Recovery_Log
    -> ORDER BY Log_ID;
+--------+----------------+-----------+-----------+-----------+-----------+-----------+
| Log_ID | Transaction_ID | Operation | Data_Item | Old_Value | New_Value | Status    |
+--------+----------------+-----------+-----------+-----------+-----------+-----------+
|      1 | T1             | START     | A         |      NULL |      NULL | ACTIVE    |
|      2 | T1             | UPDATE    | A         |       100 |       150 | ACTIVE    |
|      3 | T2             | START     | B         |       200 |       250 | ACTIVE    |
|      4 | T2             | UPDATE    | B         |       200 |       250 | ACTIVE    |
|      5 | T1             | COMMIT    | A         |       100 |       150 | COMMITTED |
|      6 | T3             | START     | C         |       300 |       350 | ACTIVE    |
|      7 | T3             | UPDATE    | C         |       300 |       350 | ACTIVE    |
|      8 | T2             | COMMIT    | B         |       200 |       250 | COMMITTED |
|      9 | T4             | START     | D         |       400 |       450 | ACTIVE    |
+--------+----------------+-----------+-----------+-----------+-----------+-----------+
9 rows in set (0.00 sec)
```

Example

<T1 START>

<T1, A, 100, 150>

<T2 START>

<T2, B, 200, 250>

<T1 COMMIT>

System Crash

At the time of crash:

T1 → Committed

T2 → Uncommitted

Recovery

T1 has successfully committed before the failure, so its changes must be preserved by applying the logged updates during recovery.

```
mysql> SELECT DISTINCT Transaction_ID
    -> FROM Deferred_Recovery_Log
    -> WHERE Status = 'COMMITTED';
+----------------+
| Transaction_ID |
+----------------+
| T1             |
| T2             |
+----------------+
2 rows in set (0.00 sec)

mysql> SELECT DISTINCT Transaction_ID
    -> FROM Deferred_Recovery_Log
    -> WHERE Status = 'ACTIVE';
+----------------+
| Transaction_ID |
+----------------+
| T1             |
| T2             |
| T3             |
| T4             |
+----------------+
4 rows in set (0.00 sec)
```

Therefore:

REDO T1 because its committed changes must be reflected in the database.

T2 did not commit before the crash.

Therefore:

IGNORE T2 because its deferred changes were never installed in the permanent database.

There is no need for UNDO because an uncommitted transaction has not written its updates to the database.

```
mysql> SELECT
    ->     Transaction_ID,
    ->     Data_Item,
    ->     Old_Value,
    ->     New_Value
    -> FROM Deferred_Recovery_Log
    -> WHERE Status = 'ACTIVE'
    -> AND Operation = 'UPDATE';
+----------------+-----------+-----------+-----------+
| Transaction_ID | Data_Item | Old_Value | New_Value |
+----------------+-----------+-----------+-----------+
| T1             | A         |       100 |       150 |
| T2             | B         |       200 |       250 |
| T3             | C         |       300 |       350 |
+----------------+-----------+-----------+-----------+
3 rows in set (0.00 sec)
```

Rule

Committed → REDO

Uncommitted → IGNORE

Why NO-UNDO?

The method is called NO-UNDO/REDO because uncommitted changes are kept outside the permanent database, so recovery only needs to redo the committed transactions whose updates may still need to be installed.

Advantages

- Recovery is simpler because incomplete database changes do not need to be reversed.
- No undo operation is required for transactions that fail before commitment.
- The recovery process has fewer cases to analyze, reducing recovery complexity.
- The database remains consistent because only committed updates are applied permanently.

9. Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Consider a banking system in which two accounts are maintained and concurrent transactions must preserve the correctness of the balances.

Account A = ₹20,000

Account B = ₹10,000

Transaction T1 transfers ₹5,000 from Account A to Account B. The transfer must be treated as one consistent transaction so that another transaction cannot observe an intermediate state.

Transaction T2 only reads the balance of Account A. Under Strict 2PL, it must not read A while T1 is holding a conflicting exclusive lock.

```
mysql> SELECT * FROM Bank_Accounts;
+------------+--------------+----------+
| Account_ID | Account_Name | Balance  |
+------------+--------------+----------+
|        101 | Anil         | 10500.00 |
|        102 | Bindu        | 15000.00 |
|        103 | Charan       | 20000.00 |
|        104 | Divya        | 12000.00 |
+------------+--------------+----------+
4 rows in set (0.00 sec)
```

T1 – Transfer Transaction

LOCK-X(A)

LOCK-X(B)

READ(A)

A = A - 5000

WRITE(A)

READ(B)

B = B + 5000

WRITE(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

After T1:

A = ₹15,000

B = ₹15,000

T2 – Read Transaction

LOCK-S(A)

READ(A)

COMMIT

UNLOCK(A)

Concurrent Execution

```
mysql> SELECT Account_ID, Account_Name, Balance
    -> FROM Bank_Accounts
    -> WHERE Account_ID = 101;
+------------+--------------+---------+
| Account_ID | Account_Name | Balance |
+------------+--------------+---------+
|        101 | Anil         | 9500.00 |
+------------+--------------+---------+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM Bank_Accounts
    -> WHERE Account_ID IN (101,102);
+------------+--------------+----------+
| Account_ID | Account_Name | Balance  |
+------------+--------------+----------+
|        101 | Anil         |  9500.00 |
|        102 | Bindu        | 16000.00 |
+------------+--------------+----------+
2 rows in set (0.00 sec)
```

Initially, T1 obtains exclusive locks on both accounts because it will modify their balances. These locks are retained until T1 completes.

X-lock(A)

X-lock(B)

T2 tries to obtain a shared lock on A so that it can read the balance.

S-lock(A)

The shared lock requested by T2 conflicts with T1's exclusive lock. Therefore, T2 cannot read the account while the transfer is in progress.

Therefore:

T2 → WAIT

T1 completes the transfer:

A = ₹15,000

B = ₹15,000

T1 commits and releases the locks. Only after this point can T2 obtain the shared lock and continue.

Then T2 obtains the shared lock and reads the committed balance:

A = ₹15,000

Why Strict 2PL is Suitable?

Strict 2PL requires exclusive locks to remain until commit or abort. This prevents other transactions from observing data while it is being modified.

It prevents:

- Dirty reads.
- Lost updates.
- Inconsistent balances.
- Reading partially completed transfers.

Banking Benefit

Strict 2PL ensures that a customer does not observe an intermediate account balance during a fund transfer. The reader sees either the previously committed state or the fully committed new state, thereby preserving transaction consistency.

10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Heuristic query optimization uses predefined transformation rules to improve a query plan without calculating the exact cost of every possible execution strategy. The rules aim to reduce intermediate relations and move data-reducing operations as early as possible.

Important Heuristics

1. Push selection down

Selection conditions should be applied as close as possible to the base relation so that irrelevant tuples are removed before joins.

```
mysql> SELECT * FROM Students;
+------------+--------------+------------+------+-------------+
| Student_ID | Student_Name | Department | CGPA | Hash_Bucket |
+------------+--------------+------------+------+-------------+
|        101 | Anil         | CSE        | 8.50 |           1 |
|        102 | Bindu        | CSE        | 9.10 |           2 |
|        103 | Charan       | CSE        | 8.90 |           3 |
|        104 | Divya        | EEE        | 7.80 |           4 |
|        105 | Farah        | ECE        | 8.20 |           0 |
|        106 | Gopal        | ECE        | 7.50 |           1 |
|        107 | Harini       | CSE        | 9.30 |           2 |
+------------+--------------+------------+------+-------------+
7 rows in set (0.00 sec)
```

2. Perform projection early

Projection should remove attributes that are not needed for selections, joins, or the final result. This reduces the width of intermediate relations.

3. Order joins carefully

Joins should be arranged so that smaller intermediate results are generated earlier, reducing memory use and later processing.

```
mysql> SELECT Student_Name, CGPA
    -> FROM Students
    -> WHERE Department = 'CSE'
    -> AND CGPA > 8.5;
+--------------+------+
| Student_Name | CGPA |
+--------------+------+
| Bindu        | 9.10 |
| Charan       | 8.90 |
| Harini       | 9.30 |
+--------------+------+
3 rows in set (0.00 sec)

mysql> SELECT Student_Name, CGPA
    -> FROM Students
    -> WHERE Department = 'CSE'
    -> AND CGPA > 8.5;
+--------------+------+
| Student_Name | CGPA |
+--------------+------+
| Bindu        | 9.10 |
| Charan       | 8.90 |
| Harini       | 9.30 |
+--------------+------+
3 rows in set (0.00 sec)
```

Pseudocode

Algorithm Optimize_Query(Query)

BEGIN

Parse Query

Create initial query tree

Identify all selections

Identify all projections

Identify all joins

// Push selections down

FOR each selection condition

Move selection close to its base relation

END FOR

// Push projections down

FOR each relation

Keep only required attributes

END FOR

// Optimize join order

Estimate size of intermediate results

Select the smallest relation

Join it with the most suitable relation

Repeat until all relations are joined

Remove unnecessary operations

Generate optimized query tree

Return optimized query tree

END

```
mysql> EXPLAIN
    -> SELECT Student_Name, CGPA
    -> FROM Students
    -> WHERE Department = 'CSE'
    -> AND CGPA > 8.5;
+----+-------------+----------+------------+------+----------------+----------------+---------+-------+------+----------+-------------+
| id | select_type | table    | partitions | type | possible_keys  | key            | key_len | ref   | rows | filtered | Extra       |
+----+-------------+----------+------------+------+----------------+----------------+---------+-------+------+----------+-------------+
|  1 | SIMPLE      | Students | NULL       | ref  | idx_department | idx_department | 123     | const |    4 |    33.33 | Using where |
+----+-------------+----------+------------+------+----------------+----------------+---------+-------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)
```

Example

Suppose:

R = 10,000 tuples

S = 500 tuples

T = 2,000 tuples

Instead of:

R ⋈ S ⋈ T

the optimizer can first perform:

S ⋈ T

if that produces a smaller intermediate result.

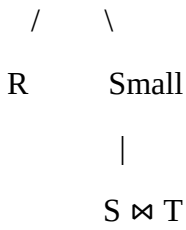
```
mysql> SHOW INDEX FROM Students;
+----------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table    | Non_unique | Key_name                 | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+----------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| students |          0 | PRIMARY                  |            1 | Student_ID  | A         |           7 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| students |          1 | idx_department           |            1 | Department  | A         |           3 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
| students |          1 | idx_student_id           |            1 | Student_ID  | A         |           7 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| students |          1 | idx_student_hash         |            1 | Hash_Bucket | A         |           5 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
| students |          1 | idx_single_student       |            1 | Student_ID  | A         |           7 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| students |          1 | idx_students_dept_cgpa   |            1 | Department  | A         |           3 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
| students |          1 | idx_students_dept_cgpa   |            2 | CGPA        | A         |           7 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
+----------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
7 rows in set (0.04 sec)

mysql> EXPLAIN
    -> SELECT Student_Name, CGPA
    -> FROM Students
    -> WHERE Department = 'CSE'
    -> AND CGPA > 8.5;
+----+-------------+----------+------------+------+-----------------------------------------+----------------+---------+-------+------+----------+-------------+
| id | select_type | table    | partitions | type | possible_keys                           | key            | key_len | ref   | rows | filtered | Extra       |
+----+-------------+----------+------------+------+-----------------------------------------+----------------+---------+-------+------+----------+-------------+
|  1 | SIMPLE      | Students | NULL       | ref  | idx_department,idx_students_dept_cgpa | idx_department | 123     | const |    4 |    33.33 | Using where |
+----+-------------+----------+------------+------+-----------------------------------------+----------------+---------+-------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)
```

Then:

(Remain Result) ⋈ R

Optimized Processing

Final Join



Benefits

- Produces smaller intermediate relations and reduces the amount of data carried through the query tree.
- Requires less memory to store and process intermediate results.
- Reduces disk I/O because fewer pages and tuples need to be processed.
- Allows join operations to work on smaller inputs and therefore improves execution speed.
- Improves the overall performance of complex SQL queries while preserving their logical result.
- Can also reduce CPU work by eliminating unnecessary tuples and attributes early.